SECURITY AUDIT

Symbiotic

Vaults V2

FINAL REPORT

June 2026

BAILSEC.IO

Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Symbiotic – Vaults V2 – Audit Report
Website	symbiotic.fi
Language	Solidity
Methods	Manual Analysis
Github repository	github.com/symbioticfi/core-mirror/blob/b2b190e8ee37cbac3d07a7d27216b4706a086baa/src/contracts
Resolution 1	https://github.com/symbioticfi/core-mirror/tree/806da483be38a6ed1c17050153e6f450f7d2448c/src/contracts
Resolution 2	https://github.com/symbioticfi/core-mirror/tree/5cba0844ba143db893692bb9079e642a042ed5d7

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High	3	2	1			
Medium	8	1		7		
Low	29	1		28		
Informational	19	3		16		
Governance	1			1		
Total	60	7	1	52		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

VaultV2

VaultV2 is the main ERC4626 vault and accounting boundary. It accepts a standard ERC20 asset, issues checkpointed vault shares, accrues management, performance, and protocol fees, and routes all adapter movement through a single UniversalDelegator. Vault assets are accounted as free collateral held directly by the vault plus allocated collateral reported by the delegator, with pending fee accrual included in totalAssets.

Deposits accrue fees, mint shares, increase cached _totalAssets, and call UniversalDelegator.onDeposit so pending withdrawals can be swept before idle collateral is auto-allocated. Withdrawals accrue fees, prioritize the WithdrawalQueue, deallocate through the delegator when free assets are insufficient, transfer assets, and decrease cached _totalAssets.

The vault deploys its WithdrawalQueue during initialization. Users can redeem instantly only after queue priority is satisfied, while the queue itself can redeem held shares without triggering the instant-withdrawal recursion guard. The delegator is bound later through the one-shot setDelegator function and must be a registered delegator entity initialized for this vault.

Appendix: Fee Accounting

getAccrueInterest previews current total assets, elapsed-time management fees, positive-yield performance fees, and cached protocol fees. accrueInterest stores the new total asset value, mints fee shares to fee receivers, updates lastUpdate, and refreshes protocol fee settings for the next accrual period.

Management fees are based on current total assets and elapsed time. Performance fees are based only on positive interest, measured as the increase from cached _totalAssets to the newly computed total asset value. Protocol fees use the cached protocol receiver and fee rates from the previous registry update.

Appendix: Delegator Pull and Push

The vault does not call adapters directly. The configured delegator is the only address allowed to call pull and push.

pull accrues interest and transfers vault collateral to the requested receiver. The delegator uses it to fund adapter allocation.

push transfers collateral from an owner address into the vault. The delegator uses it after adapter deallocation, relying on the adapter's approval to let the vault pull returned collateral.

Privileged Functions

- pull
- push
- setDepositWhitelist
- setDepositorWhitelistStatus
- setIsDepositLimit
- setDepositLimit
- setManagementFee
- setPerformanceFee
- setDelegator
- setSlasher

Core Invariants:

INV 1: totalAssets must equal vault free collateral plus assets reported by the UniversalDelegator, adjusted for fee accrual.

INV 2: Only the configured delegator can move collateral through pull and push.

INV 3: The delegator can only be set once and must be a valid registered delegator bound to this vault.

INV 4: Instant withdrawals must not bypass pending WithdrawalQueue requests.

INV 5: Fee shares must be minted from the same accounting snapshot used to update _totalAssets.

INV 6: Deposit whitelist and deposit limit settings must be enforced before shares are minted.

INV 7: Checkpointed balances and total supply must remain consistent with ERC20 transfers, mints, and burns.

INV 8: WithdrawalQueue redemptions must be allowed to redeem queued shares without recursively blocking on the same queue-priority rule.

INV 9: _totalAssets must increase by deposited assets, decrease by withdrawn assets, and recognize adapter gains or losses through fee accrual.

Issue_01	Zero initial depositor can permanently brick whitelisted deposits
Severity	**Governance**
Description	VaultV2 initialization can whitelist address(0) as the initial depositor. If repair or migration roles are also unset or unavailable, the depositor whitelist can remain in a state where no real account is permitted to deposit. This is a deployment-parameter risk rather than an external exploit, but it can permanently prevent deposits for a misconfigured vault.
Recommendations	Reject address(0) as the initial depositor when depositor whitelisting is enabled. Alternatively, require a valid repair or administrative role during initialization so that a bad whitelist entry can be corrected.
Comments / Resolution	Acknowledged.

Issue_02	Vault composition can be overridden via donation attacks
Severity	**High**
Description	*[Note: Morpho V1 vaults have a bug that makes them susceptible to this same attack vector, which was exploited recently, leading to 8-figure losses. Full report of the attack can be found here and here.]* The vault and the adapters use raw balanceOf to check their assets in a particular market. This makes them susceptible to donations. In fact, users can donate funds to the vault in bad markets, allowing them to withdraw funds from good ones due to this nature. Assume the vault contains 2 adapters, one for aave v3, and one for a morpho vault, which is now illiquid due to some depeg event (say). The vault currently has all its assets in the Aave adapter, and absoluteLimitOf and shareLimitOf are both set to 0 for the morpho adapter. The morpho adapter has a fei wei of liquidity, because of which the adapter cannot be fully removed. The vault is entirely unaffected, since all its funds are in the aave adapter and the bad adapter's limits are set to 0, so no funds should be flowing into it. However, there exists an exploit path where the vault will be forced into the vault adapter. Say Alice is the sole holder of the vault, with 1e18 tokens and 1e18 share tokens. All except a few wei are in the aave adapter, so Alice can remove ~1e18 tokens if required. Bob deposits 1e18 tokens to the vault. Bob gets minted 1e18 shares. Aave adapter now holds 2e18 tokens, and the morpho adapter has a few wei. Bob then donates 2e18 morpho vault shares to the morpho adapter. Now the totalAssets of the vault is 2e18 on Aave + 2e18 on morpho = 4e18. The vault cannot prevent this since it's a direct donation. Bob now redeems their 1e18 shares. They get back 2e18 tokens, all from

	the Aave adapter. Now the vault is left in a state where it holds 2e18 tokens in a morpho vault, which is illiquid, and Alice is now stuck. This happened despite the vault specifically preventing allocation into that adapter. Bob received 1e18 tokens extra (from the Aave adapter) and donated 2e18 morpho vault shares. Since the vault is illiquid, Bob could have purchased vault shares at a lower price, leading to a profitable attack. The same attack was carried out on multiple Morpho V1 vaults during the USR/RLP depeg. Morpho V2 vaults prevent this by tracking assets via a storage variable instead of raw balanceOf.
Recommendations	Track assets via a storage update instead of a simple balanceOf.
Comments / Resolution	Partially Fixed. Morpho adapter now tracks balance using storage variable. Other adapters however still use the same balanceOf function to track assets.

Issue_03	Vault is susceptible to inflation attacks via donations
Severity	**Medium**
Description	The adapters and the vault measure their totalAssets via a raw balanceOf check. This makes them susceptible to donations, and the first depositor can manipulate the share ratio to cause losses to other depositors by minting 0 shares to them and claiming their funds. Say a fresh vault is deployed, with an asset with 18 decimals. Then decimalsOffset = 0. Alice deposits 1 wei of assets and gets minted 1x1/1 = 1 share. Alice then donates 2e18 assets to the vault. Bob tries to mint 1e18 assets. He then gets minted 1e18 x (2)/(2e18+1) = 0 shares. Since the deposit function does not prevent 0 share mints, Bob will be minted 0 shares. Due to the decimal offset, Alice can only claim half of her input funds, so she does not make a profit unless Bob makes the same mistake twice. However, there are multiple impacts of this issue: 1. Bob did take a loss. He lost his entire deposit. 2. Alice can keep inflating the share ratio. The vault now has 3e18+1 assets. So if Alice deposits 1.5e18 tokens again, she gets minted 1.5e18 x 2 / (3e18+1) =0 shares. So now each share is worth 4.5e18 assets. Alice can keep donating 1 wei less than half of the totalAssets and mint herself 0 shares, inflating the ratio. At the end of the day, the vault can become unusable because the share ratio is so high that users are unable to mint any shares with a realistic amount of funds, bricking the vault.
Recommendations	Consider implementing the following: 1. Block deposits/mints of 0 shares 2. Tracking the assets internally by updating a storage value in the vault instead of using raw balanceOf 3. Using a base decimals offset of 21. This way, 18 decimal tokens will have a 21 decimal share, providing them with a higher amount

	of protection from such attacks.
Comments / Resolution	Acknowledged.

Issue_04	Post-Insolvency Deposits Mint Excessive Shares and Break Vault Accounting
Severity	Low
Description	When a vault loses all of its assets through slashing or other losses, VaultV2 updates totalAssets to zero but does not reduce totalSupply. Existing shares stay in circulation even though they no longer represent meaningful asset value. The problem is that losses remove assets from adapters without burning vault shares, and the vault has no handling for the dead state where totalAssets == 0 while totalSupply > 0. If anyone deposits after that point, standard ERC4626 math applies. With zero assets and a non-zero supply, each deposit mints shares equal to roughly deposit amount × totalSupply. Share counts grow far beyond what the deposited assets justify, and legacy holders keep their old shares on a broken basis. This can make the vault unrecoverable. Once share counts grow large enough, later deposits can revert from overflow. There is no in-contract reset, so the vault may never return to normal operation without deploying a new one. Recovery deposits also fail legacy depositors. If a curator deposits 1M assets to recapitalize after a total loss, original holders with zombie shares receive only a tiny fraction of that recap while most value goes to the new depositor. The vault may hold assets again, but share pricing no longer reflects fair ownership. WithdrawalQueue and other integrations that rely on sane share-to-asset ratios can also break. Absurd share counts can cause incorrect fill caps, rounding errors in claim accounting, and broken behavior anywhere the vault is treated as a normal ERC4626 token. Example: A vault holds 1,000 ETH from many depositors (~1e21 shares). After one or more slashes under normal operation, all collateral is gone. totalAssets becomes 0, but ~1e21 shares remain. The next deposit of 1 ETH mints about 1e21 new shares. One ETH creates a share count on the same order as the entire pre-loss supply. If deposits continue after further losses, each new deposit multiplies against the already inflated

	supply. With deposit sizes around 1e18 units, a few post-wipeout deposits can push totalSupply toward uint256 limits and cause later deposits to revert. A recap deposit of 1M assets after total loss leaves legacy holders with only a tiny share of that recovery because their zombie shares still count against the new deposit.
Recommendations	Consider blocking deposits and mints when totalAssets == 0 and totalSupply > 0, and add a controlled recovery path for total insolvency, such as admin-triggered share reset or migration to a new vault with a defined snapshot. Also, clearly document this edge case and its recovery path.
Comments / Resolution	Acknowledged.

Issue_05	Instant withdrawals can frontrun loss-realization
Severity	**Low**
Description	The vault allows instant withdrawals via the direct withdraw function. Users can frontrun transactions that carry out slashing or realize bad debt in the adapters, and just avoid taking on any loss. AppAdapter.totalAssets() counts slashable stake at full value until an authorized slash() call is executed. If a slash is known or expected but not yet executed, liquid users can withdraw from other vault liquidity while the slashable position still appears fully valuable in vault accounting. Remaining holders can then absorb the realized loss when slash() is eventually called. This creates a frontrunning and stale-accounting window around known slashing events.
Recommendations	Consider acknowledging this issue or enforcing a withdrawal delay for all withdrawals.
Comments / Resolution	Acknowledged.

Issue_06	Official withdrawal queue cannot be migrated through the vault
Severity	**Low**
Description	VaultV2 creates its official WithdrawalQueue with the vault itself as owner. The vault does not expose a forwarder or replacement hook that lets governance migrate that queue through WithdrawalQueueFactory. If a live queue implementation needs to be upgraded or replaced, the owner relationship can prevent the normal factory migration flow unless a separate migration path already exists outside the vault.
Recommendations	Add a controlled vault function to migrate or replace the official withdrawal queue. Alternatively, document that the queue implementation is immutable for a live vault and cannot be upgraded through the standard factory migration path.
Comments / Resolution	Acknowledged.

Issue_07	Withdrawers can manipulate the vault composition
Severity	**Low**
Description	When withdrawals are processed, they are done so in the order of the adapters present in the adapters array. The issue with this is that it withdraws funds from the first vault first and the last vaults last. Say the vault has multiple adapters, and the allocation is arranged in order of safety, with safer vaults in the beginning and more volatile vaults in the end, with more restrictions and a lower shareLimitOf limits. When a large deposit comes in, it can fill all the vaults, including the riskier lower limit ones. But then, when funds are withdrawn, they are done so according to the adapters array order, which can empty out the safe adapters and leave funds in the riskier ones. A user can take a large flashloan, deposit funds, and then immediately withdraw most of them, leaving the vault with a very high percentage of funds in the very last adapter (in their adapters array). Thus, external users can manipulate the composition to some extent, which might mess with the risk profile of the vault.
Recommendations	Consider acknowledging this issue and having this in the risk model of the vault. Another option would be to withdraw funds proportionately from each adapter, mitigating this to some extent.
Comments / Resolution	Acknowledged.

Issue_08	Zero-value ERC4626 calls can trigger auto-allocation
Severity	**Low**
Description	VaultV2 calls the delegator onDeposit() from the internal deposit path. Zero-asset deposit calls or zero-share mint calls can still reach this hook, allowing a caller with no capital at risk to trigger automatic allocation of existing idle vault assets into configured adapters. This can change vault liquidity and risk exposure at a time chosen by a zero-capital caller.
Recommendations	Skip delegator onDeposit() when the deposited asset amount is zero. Alternatively, reject zero-asset deposits and zero-share mints at the public ERC4626 entry points.
Comments / Resolution	Acknowledged.

Issue_09	Public one-shot delegator binding can capture directly-created vaults
Severity	**Low**
Description	VaultV2 exposes setDelegator() as a public one-shot binding hook. Factory-created vaults are expected to bind their intended delegator during setup, but a VaultV2 deployed directly and left unbound can be irreversibly bound by any caller to an attacker-controlled delegator. The attacker would then control the vault allocation path. This is mainly a deployment-safety issue because the standard factory flow should bind the correct delegator immediately.
Recommendations	Restrict setDelegator() to the factory, owner, or an initialization-only caller. Alternatively, document that VaultV2 must not be deployed directly unless setDelegator() is called atomically in the same setup flow.
Comments / Resolution	Acknowledged.

Issue_10	Long-unaccrued fees can exceed the current NAV and brick accrual
Severity	**Low**
Description	VaultV2 computes management, performance, and protocol fee assets before subtracting them from the current total assets to derive the post-fee asset base. If a vault remains unaccrued for long enough, or if fee settings are high relative to remaining NAV after losses, the calculated fees can exceed current assets and make the subtraction underflow. Because deposits, withdrawals, redeems, and fee updates depend on accrueInterest(), this can turn an excessive stale fee window into a denial of service for normal vault operations.
Recommendations	Cap accrued fee assets to the available feeable NAV before subtraction, or make the accrual path saturate and emit the maximum collectible fee instead of reverting. Also consider enforcing fee-rate bounds that make this state unreachable under expected accrual intervals.
Comments / Resolution	Acknowledged.

Issue_11	Performance fees can be charged on recovery from prior losses
Severity	**Low**
Description	VaultV2 accrues performance fees from any positive increase in total assets since the last fee checkpoint. If the vault first suffers a loss or slash and later recovers part of that value, the recovery can be treated as positive performance even though holders have not received net new profit relative to the pre-loss state. This dilutes existing holders during loss recovery. The practical severity depends on the intended fee model: if fees are meant to apply to every positive interval, this is expected behavior. If fees are meant to apply only to net new profit, the vault is missing a high-water mark or loss carry-forward.
Recommendations	Consider acknowledging this fee model explicitly. If performance fees should only apply to net profit, track a high-water mark or loss carry-forward, and exclude recovery from prior losses or slashes from feeable performance.
Comments / Resolution	Acknowledged.

Issue_12	Management fees use ending NAV for the full elapsed interval
Severity	**Low**
Description	VaultV2 management fees are calculated from the current ending NAV over the elapsed accrual interval. If yield, rewards, or recovered assets arrive late in that interval, they can be charged as if they were managed for the entire period. This can make fee dilution depend on accrual timing rather than only on time under management. In fact, the less frequently the accrual function is called, the higher the management fees charged since the fee depends on the final amount x time elapsed. Furthermore, because accrueInterest() is permissionless and fee calculations round down per accrual window, very frequent accrual can also reduce collected fees. Small management or performance fee amounts can round to zero, and the skipped fractional fees are not accumulated for later collection. This mainly affects fee recipients in small or low-decimal vaults and is bounded by gas costs and rounding dust.
Recommendations	Consider acknowledging this fee model. If timing neutrality is desired, accrue fees before large known NAV changes, or track time-weighted asset balances instead of charging the full elapsed interval against ending NAV.
Comments / Resolution	Acknowledged.

Issue_13	No pause mode
Severity	**Low**
Description	The Vault does not implement a pause mode to easily block deposits in case of any issues.
Recommendations	Consider adding a pause functionality so that the vault operations can be quickly halted if required.
Comments / Resolution	Acknowledged.

Issue_14	Hooked-asset reentrancy on unguarded vault flows
Severity	**Low**
Description	Vault deposit/withdraw/redeem carries no nonReentrant guard; with a hooked asset (ERC777/ERC1363/callback token), a transfer callback can reenter, and specifically _totalAssets -= assets runs AFTER the external transfer, so a reentrant accrueInterest could double-decrement the cached total and over-charge the performance fee.
Recommendations	Do not support tokens with hook callbacks. Otherwise, all functions need to be guarded with the nonreentrant modifier.
Comments / Resolution	Acknowledged.

Issue_15	VaultV2 does not override maxWithdraw() and maxRedeem()
Severity	**Low**
Description	VaultV2 is an ERC4626 vault. However, maxWithdraw() and maxRedeem() are not overridden. Therefore, they use the default ERC4626 behavior, where maxRedeem() returns the user's share balance and maxWithdraw() returns the user's shares converted to assets. This is not necessarily correct for VaultV2, because the vault may have fewer currently withdrawable assets than the user's full claim. Assets can be allocated into adapters, pending delayed deallocation, or otherwise unavailable for immediate withdrawal. In such cases, maxWithdraw() / maxRedeem() can overstate what the user can actually withdraw or redeem.
Recommendations	Consider acknowledging and documenting this issue.
Comments / Resolution	Acknowledged.

Issue_16	Missing Slippage Protection on Vault Deposits
Severity	**Low**
Description	VaultV2 follows the standard ERC4626 deposit and mint. Users set an asset or share amount, but cannot set a minimum acceptable output on-chain. There is no check that the result matches what they expected from a prior quote. On deposit, the vault accrues interest, mints shares at the current rate, updates accounting, and then allocates assets through UniversalDelegator.onDeposit. Shares are fixed before allocation ends. If allocation into Morpho, Aave, or other adapters takes a loss in the same transaction, from fees, rounding, or a worse external price, true NAV can fall after shares are already minted. A user may deposit 100 assets, receive shares priced on pre-allocation NAV, and hold less value once allocation completes. The same risk exists across transactions. State can change between quote and execution through other deposits, withdrawals, interest accrual, adapter losses, or fee minting. The vault still processes the deposit at the new rate, which may be worse than intended. This matches common ERC4626 behavior and is not a critical failure. Users can cap risk off-chain via mint with a max assets check or post-deposit balance checks. But the vault offers no native slippage guard, and deposit-then-allocate ordering makes entry losses possible in one transaction.
Recommendations	Consider adding new variants of the deposit/mint functions that provide optional slippage limits, and revert when the outcome is worse than the user allows. Alternatively, require integrators to enforce these checks in a wrapper.
Comments / Resolution	Acknowledged.

Issue_17	maxDeposit() is caller-dependent when the depositor whitelist is enabled
Severity	Informational
Description	VaultV2.maxDeposit(receiver) checks isDepositorWhitelisted[msg.sender] and ignores the receiver parameter. ERC4626 defines maxDeposit(receiver) as a receiver-specific deposit capacity quote. As a result, different callers can receive different maxDeposit values for the same receiver. This can confuse frontends, routers, and off-chain integrations that query maxDeposit(user), expecting the result to depend on that user. The deposit path calls maxDeposit(receiver) in the same caller context, so a non-whitelisted caller cannot bypass the whitelist for positive deposits. The issue is limited to ERC4626 compatibility and integration correctness.
Recommendations	Document that depositor whitelist limits are caller-specific, or expose a separate caller-aware deposit limit view for integrations.
Comments / Resolution	Acknowledged.

Issue_18	Fee receivers are paid in shares and remain exposed to vault PnL
Severity	**Informational**
Description	VaultV2 pays management, performance, and protocol fees by minting vault shares. Those shares participate in later yield, losses, and slashes like any other holder position. This means fee receivers are not paid a fixed asset amount at accrual time. They remain exposed to subsequent vault performance unless they redeem their fee shares.
Recommendations	Consider acknowledging this design. If fee receivers should avoid post-accrual vault exposure, pay or immediately distribute fees in assets instead of shares.
Comments / Resolution	Acknowledged.

Issue_19	redeemable() can overstate safely redeemable shares due to rounding
Severity	**Informational**
Description	VaultV2.redeemable() converts withdrawable assets into shares with previewWithdraw(). Since previewWithdraw() rounds shares up, the result can be slightly higher than the shares that can actually be redeemed through the real instant withdrawal path. This can cause integrators or frontends to display optimistic redeem capacity and submit redeem attempts that revert or need to be retried with a lower amount.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Fixed.

Issue_20	redeemable() overstates instant-withdrawable liquidity
Severity	Informational
Description	redeemable, and the withdrawable it builds on, reports the gross instant liquidity: the vault's idle assets plus the instantly-deallocatable adapter assets. But the withdrawal queue has strict priority over instant withdrawals, because an instant withdraw or redeem first runs sweepPending and reverts entirely if the queue cannot be fully filled. A new instant withdrawer can therefore only ever access withdrawable minus whatever the queue consumes first, and when the queue is not fully fillable, the true instant-redeemable amount is zero while redeemable still returns a large positive figure.
Recommendations	Consider acknowledging and documenting this issue.
Comments / Resolution	Acknowledged.

Issue_21	totalSupplyAt can be incorrect due to non-accrual of fees
Severity	Informational
Description	The totalSupplyAt function is supposed to reflect the totalSupply of the share tokens at a given point in time. However, since this function does not accrue interest, it will be incorrect if the current timestamp is used. Basically, totalSupply() returns a different value than totalSupplyAt(block.timestamp), even though they measure the same thing at the same timestamp. The same pending fee shares are also not reflected in balanceOf() or balanceOfAt() until accrueInterest() mints them. This means totalSupply() can temporarily exceed the sum of visible balances, and fee receivers may not be able to transfer their full accrued fee shares before accrual.
Recommendations	Consider acknowledging this issue and documenting that totalSupplyAt(), balanceOf(), and balanceOfAt() may not include pending fee share accrual until accrueInterest() is called.
Comments / Resolution	Acknowledged.

WithdrawalQueue

WithdrawalQueue holds pending vault share redemptions as ERC721 positions. Users transfer vault shares into the queue, receive a withdrawal NFT, and later claim underlying assets as the queue is filled. The queue separates the act of requesting redemption from the act of receiving assets, which lets VaultV2 prioritize pending exits before allowing instant withdrawals.

Appendix: Request, Fill, and Claim Mechanism
requestRedeem records the requested shares, mints the NFT to the receiver, increments totalRequested, and triggers UniversalDelegator.sweepPending through the vault. The NFT owner is the recipient of future claim proceeds, so withdrawal positions can be transferred before they are fully claimable.

fill is permissionless. It computes pending shares, caps the fill by the vault's current withdrawable assets, redeems the queue's vault shares into underlying assets, and records a cumulative filled-shares-to-assets checkpoint. Those checkpoints define the exchange curve used to split filled assets across requests in request order.

claimable uses the request's share range and the cumulative checkpoints to compute newly claimable assets. claim updates sharesClaimed and transfers assets to ownerOf(tokenId).

Core Invariants:

INV 1: totalRequested must equal the sum of all queued withdrawal shares.

INV 2: pendingShares must equal totalRequested minus totalFilled.

INV 3: totalFilled must only increase when vault shares are redeemed into assets.

INV 4: A request can only claim filled shares that were not already claimed.

INV 5: Claim proceeds must be paid to the current withdrawal NFT owner.

INV 6: Fill checkpoints must remain cumulative and monotonic in shares and assets.

Issue_22	Pending queue can be griefed by any user
Severity	Medium
Description	The fill function fills queued withdrawals using free tokens present in the vault contract. This function can be triggered by anyone, either via a direct call or via calling sweepPending. The issue is that due to rounding, 0 assets can be sent to the queue while burning a non-zero amount of shares. This can be deliberately triggered in the following way. Assume a queue exists. The vault contains 90 tokens and has minted 100 shares. This can happen after a bad debt realization or slashing. Assume the aave adapter contains all the tokens. A user can take a flashloan from Aave, so that there's only 1 token remaining. They then call sweepPending. In sweepPending, _deallocateAll sends that 1 token to the vault and calls fill. In fill(), vault.withdrawable reports 1 asset. previewDeposit then calculates this as 1x101/91 = 1 share. But if this 1 share is redeemed, 0 assets are actually received. Thus, a user can: 1. Control liquidity and thus withdrawable in the adapters using flash loans, 2. Trigger redeem calls of very specific share amounts that net 0 assets. Using this, users can grief redemption queues, burning their shares for free without giving out assets. Normally, such 0 amount withdrawals are only possible by the owner themselves, but here one user can trigger it for another user, which is an issue. The same fill-curve rounding can make claimable() return a nonzero filled share amount with zero assets, so claim() may advance sharesClaimed for a dust slice without transferring assets.

Recommendations	Revert 0 asset redeems.
Comments / Resolution	Acknowledged.

Issue_23	Direct fill() can skip AppAdapter pending-debt synchronization
Severity	**Low**
Description	WithdrawalQueue.fill() can satisfy queued withdrawals directly from available vault liquidity. That path can reduce or clear queue demand without running the same post-fill synchronization that UniversalDelegator.sweepPending() performs for delayed AppAdapter debt. If stale AppAdapter debt remains live after the queue was filled from other liquidity, it can later mature and release slashable stake even though the original pending withdrawal demand is gone. A later sweepPending() can clear this in healthy limit configurations, but the stale state can persist if no sweep occurs before maturity or if current limits prevent the reset from being requested.
Recommendations	Have direct fills notify the delegator after queue demand changes, or make sweepPending() run the same synchronization after any successful fill. Also consider letting AppAdapter clear stale debt independently of current allocation limits when pending queue demand is zero.
Comments / Resolution	Acknowledged.

Issue_24	Withdrawal NFT transfers include remaining claim rights
Severity	**Low**
Description	WithdrawalQueue positions are represented as ERC721 tokens. claim() pays assets to the current ownerOf(tokenId) and records claimed progress on the withdrawal request. If a withdrawal NFT is transferred, the new owner receives all remaining claim rights for that request. This includes assets that were already claimable but not yet claimed before the transfer, as well as assets that become claimable later.
Recommendations	Document that withdrawal NFTs transfer all unclaimed claim rights. Integrations should show sharesClaimed, total requested shares, current claimable assets, and remaining unfilled shares before allowing secondary sales or transfers.
Comments / Resolution	Acknowledged.

Issue_25	isClaimed() reports nonexistent requests as claimed
Severity	**Informational**
Description	WithdrawalQueue.isClaimed() compares the claimed share amount against the request share amount. For a nonexistent token ID, both values are zero, so the function returns true. Integrations can misclassify an invalid withdrawal request as a completed one instead of recognizing that the request does not exist.
Recommendations	Return false for nonexistent token IDs, or expose a separate exists() check and document that isClaimed() only applies to valid request IDs.
Comments / Resolution	Fixed.

WithdrawalQueueFactory

WithdrawalQueueFactory is the versioned factory for WithdrawalQueue proxy deployments. It inherits MigratablesFactory and adds no custom state or deployment logic. Its main protocol role is to let VaultV2 deploy a queue instance that is registered, initialized once, and bound to the vault that created it.

Appendix: Versioned Deployment Mechanism

The owner controls whitelisted implementations and blacklisted versions. Anyone can call create for an available version. The factory deploys a MigratableEntityProxy, registers it as an entity, and initializes it with the supplied owner and data. Versioning and migration entry points are centralized in the factory, while each queue implementation decides whether migration is actually supported.

VaultV2 uses this factory during initialization to deploy the queue for the vault, passing the vault address as the queue owner and encoded vault name and symbol metadata.

Privileged Functions

- whitelist
- blacklist

Core Invariants:

INV 1: Only the factory owner can whitelist implementations or blacklist versions.
INV 2: create must only deploy whitelisted implementations.
INV 3: Every created queue must be registered before initialization completes.
INV 4: A queue proxy must be initialized exactly once.
INV 5: The queue owner supplied by VaultV2 must become the bound vault address.

No Issues Found

UniversalDelegator

UniversalDelegator is the vault's adapter router and limit controller. It stores the ordered adapter list, auto-allocation route, per-adapter absolute limits, per-adapter share limits, and pending deallocation request state. It is the only component that decides which adapters receive new vault collateral and which adapters are asked to return collateral for withdrawals.

Appendix: Adapter Registration and Limits

Each delegator is initialized for one registered VaultV2. addAdapter only accepts adapters that are registered factory entities, initialized for the same vault, and whose factory is whitelisted for that vault in AdapterRegistry. Adapter order matters because allocation, deallocation, and delayed deallocation requests all walk configured adapter arrays.

Appendix: Allocation and Deallocation Flow

Allocation is routed through VaultV2.pull into adapters. Before allocation, the delegator calls sweepPending.If withdrawal queue assets remain pending, allocation returns zero. Auto-allocation uses the configured autoAllocateAdapters route after deposits and is capped by both delegator-side limits and adapter-side allocatable capacity.

Deallocation is routed through adapters back into VaultV2.push. onWithdraw deallocates across the ordered adapter route when instant withdrawals need liquidity. forceDeallocate attempts immediate deallocation, requests delayed deallocation for any shortfall, and lowers the adapter's absolute limit. The return value of each adapter deallocation controls how much liquidity the vault can actually recover.

Appendix: Withdrawal Queue Sweep

sweepPending is permissionless. It first asks adapters to return free assets, then deallocates enough assets to cover pending queue assets net of vault free assets, calls WithdrawalQueue.fill, and requests delayed deallocation from adapters for any remaining pending assets. This function is the bridge between queued withdrawal demand, vault free liquidity, immediate adapter liquidity, and delayed adapter release mechanics.

Privileged Functions

- addAdapter
- removeAdapter
- setLimits
- swapAdapters
- setAutoAllocateAdapters
- allocate
- allocateAll
- deallocate
- deallocateAll
- deallocateExact
- forceDeallocate
- decreaseLimits
- onDeposit
- onWithdraw

Core Invariants:

INV 1: The delegator must be bound to exactly one registered VaultV2
.
INV 2: Only vault-bound adapters from whitelisted factories can be added.

INV 3: Adapter allocation must respect absolute limits, share limits, vault free assets, and adapter allocatable capacity.

INV 4: Allocation must be skipped while withdrawal queue assets remain pending.

INV 5: Deallocation must only report assets that can be returned to the vault through VaultV2.push.

INV 6: adaptersWithPending must track adapters with active delayed deallocation requests.

INV 7: Adapters with nonzero totalAssets must not be removable.

INV 8: Configured adapters must retain delegator allocation and deallocation roles until removed.

Issue_26	Temporary deposits can bypass AppAdapter share caps
Severity	Medium
Description	UniversalDelegator.limitOf() applies shareLimitOf against the current VaultV2.totalAssets(). A temporary deposit increases current TVL and therefore raises the effective share cap used during onDeposit() allocation. An attacker can deposit, let auto-allocation route the temporarily-expanded cap into AppAdapter, and then withdraw through liquid adapters. Because AppAdapter slashable stake cannot be immediately deallocated, the attacker can exit while remaining users are left with AppAdapter exposure above the intended share cap. This defeats the share cap as a risk-control mechanism and transfers slash concentration to the remaining holders without requiring a permanent asset sacrifice in the clean flow.
Recommendations	Consider not allocating fresh deposits into slash-delayed, share-capped adapters solely because the deposit temporarily increased current TVL. For AppAdapter or any delayed-deallocation adapter, compute the share cap against a conservative denominator such as pre-deposit totalAssets(), durable non-transient TVL, or TVL excluding the current deposit until the withdrawal/deallocation risk created by that deposit can be settled. Alternatively, keep withdrawals fully available but make onDeposit() skip or cap allocations into delayed adapters whenever the allocation would leave remaining holders above the configured share cap after the depositor's own shares are removed from the denominator. Excess deposit liquidity can remain idle or route only to immediately deallocatable adapters.
Comments / Resolution	Acknowledged.

Issue_27	Pending withdrawal debt can be assigned to adapters with no-op requestDeallocate()
Severity	Medium
Description	UniversalDelegator.sweepPending() routes unresolved withdrawal queue demand through the configured adapter order. After trying to deallocate currently free or immediately available assets, it recomputes pendingAssets() and then assigns the remaining pending amount to adapters by using each adapter's totalAssets(). For each selected adapter, the delegator calls requestDeallocate(toRequest), records the adapter in adaptersWithPending, and subtracts toRequest from remainingPendingAssets. The subtraction happens regardless of whether the adapter actually schedules future liquidity. Immediate adapters such as AaveV3Adapter or MorphoVaultV2Adapter inherit the base Adapter requestDeallocate() behavior, where _requestDeallocate() is a no-op. If one of those adapters appears earlier in the deallocation route and has enough totalAssets() to absorb the pending amount, sweepPending() can treat the queue debt as assigned to that adapter even though no delayed release was recorded. As a result, a later AppAdapter may never receive the requestDeallocate() call that would start its slash-delay release window. Queued withdrawals can remain pending until new liquidity arrives, the immediate adapter becomes withdrawable, or an operator intervenes by changing configuration or force-deallocating. Example: a vault has 100 pending withdrawal assets, a first adapter with 100 totalAssets() but no currently executable liquidity and no-op requestDeallocate(), and a second AppAdapter with 100 slashable assets. sweepPending() assigns the full 100 pending amount to the first adapter, subtracts it from remainingPendingAssets, and never reaches the AppAdapter. No AppAdapter debt checkpoint is created, so the delay timer that should eventually release slashable stake never starts.

Recommendations	Only assign pending debt to adapters that can either deallocate immediately or record a meaningful delayed request. Alternatively, have requestDeallocate() return the amount of pending debt actually scheduled, and continue routing any unresolved pending demand to later adapters. Another option is to separate adapter capabilities: immediate-only adapters should not be included in delayed pending-debt assignment unless they expose an executable-liquidity amount that can actually satisfy the queue.
Comments / Resolution	Acknowledged.

Issue_28	forceDeallocate() can underflow when LiquidityLaneAdapter realizes rewards
Severity	Medium
Description	UniversalDelegator.forceDeallocate() snapshots adapter totalAssets(), caps the requested amount to that snapshot, and then calls the adapter deallocation path. After deallocation, it updates the adapter's absolute limit with: *adapterTotalAssets - deallocated - pending* This assumes deallocated plus pending cannot exceed the pre-call adapterTotalAssets snapshot. LiquidityLaneAdapter can return more than that snapshot. Its totalAssets() is freeAssets() plus allocated vault-funded principal. During _deallocate(), each redemption account returns both principal and rewards. The adapter subtracts only principal from allocated, transfers principal plus rewards back from the account, and reports principal plus rewards as deallocated. For example, if LiquidityLaneAdapter has 100 allocated principal and 10 claimable rewards in its redemption accounts, totalAssets() returns 100. forceDeallocate() snapshots 100 and requests at most 100. LiquidityLaneAdapter.deallocate() can then return 110 because it realizes both principal and rewards. The delegator limit update computes 100 - 110 - 0, which underflows. The revert prevents the emergency force-deallocation from completing and prevents the delegator from reducing the adapter's limit to quarantine the position.
Recommendations	Make the forceDeallocate() limit update saturating instead of reverting when deallocated plus pending exceeds the pre-call totalAssets snapshot. For example, compute the remaining limit base with saturating

	subtraction and set it to zero when emergency deallocation returns more than the snapshot. Also make the adapter/delegator accounting contract explicit: either totalAssets() must include all value that deallocate() can return, including claimable rewards, or forceDeallocate() must tolerate adapters returning more than their pre-call accounted principal.
Comments / Resolution	Fixed.

Issue_29	sweepPending can be abused to selectively drain adapters
Severity	Medium
Description	sweepPending is permissionless and satisfies pending withdrawals by pulling liquidity from adapters in fixed order, taking whatever each adapter can currently return. The issue is that the amount that an adapter can deallocate is actually in the user's control. For Aave and Morpho adapters, the deallocatable amount is the actual funds in the aave/morpho contract. However, users can take a flash loan from those contracts to artificially drain them, showing as 0 deallocatable. Say a withdrawal request exists in the queue of 100 tokens, and the adapters in order, along with their free liquidity, are aave (100), morpho (100), and appAdapter (100). Users can call sweepPending to drain the Aave adapter as designed. Or, they can take a flash loan from Aave and then call sweepPending, to drain the morpho adapter, since Aave will return 0 deallocatable. Or, they can take flashloans from both and queue up a deallocation from the appAdapter. Especially for the slashable adapters, users can abuse them by constantly queueing up deallocations on them by taking flashloans from the other adapters, forcing them to show 0 deallocatable, and thus triggering slashable adapter deallocations.
Recommendations	Consider acknowledging this issue. Admin must be active in re-allocating funds to AppAdapters so that too much funds are not deallocated sitting idle.
Comments / Resolution	Acknowledged.

Issue_30	_sweepPending() declared public bypasses the sweepPending() reentrancy guard
Severity	Low
Description	The UniversalDelegator splits the queue sweep into sweepPending(), which is marked nonReentrant, and _sweepPending(), which holds the actual logic. Despite being documented as internal, following the underscore naming convention, and only ever being called from inside the contract, _sweepPending() is declared public. Since the unguarded implementation is externally reachable, the reentrancy protection on sweepPending() can be bypassed by calling _sweepPending() directly, nullifying the guard that this version deliberately added. Practical exploitability is conditional today, since the asset is assumed to be a standard ERC20 and the adapters and queue are trusted, so no attacker controlled callback is reached, but the visibility is clearly unintended.
Recommendations	Declare _sweepPending() as internal.
Comments / Resolution	Fixed.

Issue_31	allocatable() can misquote capacity because it does not simulate sweepPending()
Severity	Low
Description	UniversalDelegator.allocatable() computes capacity from adapter limits, current adapter assets, vault free assets, and adapter-level capacity. Real allocation first runs sweepPending(), so the returned value is a current-state quote rather than a post-sweep executable quote. It can overstate capacity when sweepPending() consumes vault free assets to fill queued withdrawals before any new allocation is made. Automation can therefore receive an optimistic allocatable quote and then allocate less, or nothing, once the real path sweeps pending queue demand. It can also understate capacity when adapters already hold idle free assets that have not yet been swept back to the vault. Those assets are included in adapter totalAssets(), reducing limit headroom, but they are not included in VaultV2.freeAssets() until sweepPending() pulls them back. In that case the next real allocation can have more available vault free assets than allocatable() reported before the sweep.
Recommendations	Expose a separate executableAllocatable() quote that simulates the post-sweep state, including pending withdrawal consumption and sweepable adapter free assets, or document allocatable() as a raw pre-sweep capacity estimate.
Comments / Resolution	Acknowledged.

Issue_32	Direct dust can block adapter removal
Severity	Low
Description	UniversalDelegator.removeAdapter() requires the adapter's totalAssets() to be zero. If the adapter can receive direct token transfers or otherwise hold dust, a third party can keep totalAssets() nonzero and prevent removal of an unwanted adapter until the dust is cleared. This can delay governance or operator cleanup of stale adapter configurations. For AppAdapter, donated free dust can become harder to clear if a later nonzero allocation creates a new stake checkpoint using the adapter's full totalAssets(), folding the donated balance into slashable stake.
Recommendations	Provide a controlled rescue or sweep path for harmless dust before adapter removal. Alternatively, allow removal when only non-managed dust remains and separately account for or recover that dust.
Comments / Resolution	Acknowledged.

Issue_33	Adding a prefunded adapter can create a NAV jump for existing holders
Severity	**Low**
Description	UniversalDelegator.addAdapter() accepts adapters that already hold assets. Before admission, those assets are invisible to VaultV2 totalAssets(). After admission, they become part of vault NAV. A depositor who enters before the adapter is added can receive shares at the lower pre-admission NAV and then benefit from the prefunded adapter value once it is included.
Recommendations	Consider rejecting adapters with pre-existing balances during addAdapter(), or require prefunded balances to be swept, donated through a documented path, or accounted for before user deposits are open.
Comments / Resolution	Acknowledged.

Issue_34	deallocatable can give different results, depending on the gas budget
Severity	**Low**
Description	deallocatable() runs __deallocateAll through a low-level self call and decodes its result, and the per-adapter _deallocate paths in MorphoVaultV2Adapter and AaveV3Adapter wrap their external withdraw in try/catch returning 0 on any failure. Because of the 63/64 gas-forwarding rule, a caller can supply just enough gas that an inner withdraw runs out of gas; the out-of-gas is swallowed by the catch and counted as 0 deallocatable for that adapter rather than reverting, so deallocatable, and the withdrawable/redeemable values that consume it, silently understate available liquidity. Thus, the result of this function is actually dependent on the gas budget for the call, and can be unpredictable. This is bound to grief callers of these view-style paths rather than corrupt state, so Low severity.
Recommendations	Consider documenting and acknowledging this issue. Otherwise, consider bubbling failures for configured adapters, adding minimum expected allocation/deallocation checks, or using explicit failure modes that allow the delegator to distinguish “no liquidity available” from “external call failed.”
Comments / Resolution	Acknowledged.

Issue_35	sweepPending avoids a reentrancy self-DoS only by a fragile invariant
Severity	Informational
Description	Several delegator entry points are non-reentrant and call the unguarded sweepPending while still holding the lock. sweepPending calls the queue's fill, which calls back into the vault's redeem, whose _withdraw would call the non-reentrant onWithdraw. If that nested redeem ever needed onWithdraw, the call would re-enter the held lock and revert the whole operation. This does not happen today only because sweepPending pre-funds the vault's free assets before calling fill, so the nested redeem is always covered by free assets and never reaches onWithdraw. The safety is entirely implicit, any future change that lets fill draw on deallocatable beyond the prefunded free assets, weakens the prefunding, or moves the reentrancy guard would silently reintroduce a hard denial of service of all deallocate, allocate and deposit operations whenever the queue holds fillable requests.
Recommendations	No code changes are required as of now. However if any logic is changed such that onWithdraw does get called, it will lead to a DOS due to the re-entrancy guard.
Comments / Resolution	Acknowledged.

Issue_36	sweepPending() gas cost scales with adapter count
Severity	**Informational**
Description	UniversalDelegator.sweepPending() can walk the full adapter set multiple times while sweeping free assets, deallocating liquidity, filling the withdrawal queue, assigning pending requests, and reconciling adaptersWithPending. The cost can grow further when many AppAdapters are pending, because AppAdapter request accounting reads limitOf(), which can re-enter vault and delegator asset accounting. With many configured adapters, queue-processing paths such as requestRedeem() and sweepPending() can become expensive and may require careful operational limits.
Recommendations	Document adapter-count and AppAdapter-count limits for production vaults. Keep active adapter sets small enough that withdrawal-queue servicing remains comfortably within expected gas limits.
Comments / Resolution	Acknowledged.

Issue_37	Zero-amount allocation can waste gas across the auto-allocation route
Severity	**Informational**
Description	UniversalDelegator._allocate() does not return early when the capped allocatable amount is zero. It can still call VaultV2.pull(0), which accrues interest and emits an event, and then call adapter.allocate(0). When _allocateAll() continues after a zero allocation, this overhead can repeat across the remaining auto-allocation route. The impact is the avoidable gas cost, especially when many adapters are configured, or vault free assets have already been fully allocated.
Recommendations	Return early from _allocate() when the capped amount is zero. Also consider stopping _allocateAll() once no vault free assets remain available for allocation.
Comments / Resolution	Fixed.

Issue_38	swapAdapters() continues scanning after both adapters are found
Severity	**Informational**
Description	UniversalDelegator.swapAdapters() loops through the full adapter array to find both adapter indices. Once both indices have been found, the loop still continues until the end of the array. This is only a bounded gas inefficiency for an admin-only function, since the adapter list is capped and swaps should be infrequent.
Recommendations	Break out of the search loop once both adapter indices have been found.
Comments / Resolution	Acknowledged.

AdapterRegistry

AdapterRegistry is an owner-controlled whitelist for adapter factories. Whitelist status is scoped by exact vault address and adapter factory address. The registry is a configuration gate between adapter deployment and delegator usage, not a runtime risk manager.

UniversalDelegator.addAdapter checks this registry before accepting an adapter. The registry does not deploy adapters or validate implementation behavior; it only records whether a factory is approved for a specific vault. Because the lookup is vault-scoped, operators must configure the entry that matches the actual vault using the adapter.

Privileged Functions

- setWhitelistedStatus

Core Invariants:

INV 1: Only the owner can change the whitelist status.
INV 2: Whitelist status must apply to the exact vault and adapter factory pair.
INV 3: UniversalDelegator.addAdapter must reject adapters from factories not whitelisted for the delegator's vault.
INV 4: Registry approval must not be treated as a guarantee that an adapter is economically safe.

No Issues Found

ProtocolFeeRegistry

ProtocolFeeRegistry stores protocol fee settings consumed by VaultV2. It supports global fees and optional vault-specific overrides. VaultV2 reads getFee when refreshing its cached protocol fee configuration, so this registry determines protocol fee rates and receivers used for future accrual windows.

Appendix: Fee Resolution and Vault Cache

The owner controls the global receiver, global management fee, global performance fee, and per-vault override settings. Fee rates are bounded by the same maximums used by VaultV2, and nonzero fees require a nonzero receiver. This keeps the registry from configuring fee values that the vault itself would reject.

If a vault override is enabled, getFee returns the override receiver and rates. Otherwise, it returns the global configuration. Vault-specific settings can therefore replace the receiver and both fee rates without changing defaults for other vaults.

Privileged Functions

- setGlobalFee
- setGlobalReceiver
- setVaultFee

Core Invariants:

INV 1: Only the owner can change global or vault-specific protocol fee settings.

INV 2: Fee rates must not exceed VaultV2 maximum fee bounds.

INV 3: Nonzero fee rates must not be configured with a zero receiver.

INV 4: getFee must return enabled vault overrides before falling back to global settings.

INV 5: Registry updates affect VaultV2 after the vault refreshes its cached protocol fee configuration.

Issue_39	Protocol fee updates do not settle the current accrual window
Severity	**Low**
Description	VaultV2 caches protocol fee settings and only refreshes them during accrueInterest(). If ProtocolFeeRegistry values change between accruals, the next accrual can still mint protocol fees using the old cached receiver or rates before updating the cache. This can send fees for the elapsed window to an old receiver or apply old rates longer than governance or integrations expect.
Recommendations	Settle the current accrual window before applying registry updates, or make VaultV2 fetch live protocol fee settings before computing the next protocol fee mint. Alternatively, document that registry changes only affect accrual windows after the next accrueInterest() call.
Comments / Resolution	Acknowledged.

AdapterFactory

AdapterFactory is the versioned factory for one adapter family. It inherits MigratablesFactory and adds no custom state or deployment logic. Separate adapter factories can be used for different adapter implementations while sharing the same versioned deployment pattern.

Appendix: Versioned Deployment Mechanism

The owner controls whitelisted adapter implementations and blacklisted versions. Anyone can call create for an available version. The factory deploys a proxy, registers it, and initializes it with the supplied owner and adapter data. The factory also owns the migration entry point for registered adapter entities, but the created adapter still has no effect on vault assets until a delegator accepts it.

UniversalDelegator can add a factory-created adapter only if the adapter is registered by this factory, initialized for the delegator's vault, and this factory is whitelisted for that vault in AdapterRegistry. This gives the protocol two layers of control: implementation/version control at the factory and vault-specific usage control at the registry.

Privileged Functions

- whitelist
- blacklist

Core Invariants:

INV 1: Only the factory owner can whitelist implementations or blacklist versions.
INV 2: create must only deploy whitelisted implementations.
INV 3: Every created adapter must be registered before initialization completes.
INV 4: An adapter must be initialized for a registered vault before it can be used.
INV 5: AdapterRegistry must whitelist the factory for a vault before UniversalDelegator can add its adapters.

No Issues Found

Adapter

Adapter is the abstract base class for vault adapters. It handles vault binding, delegator-only access control, default free asset accounting, and shared allocation and deallocation entry points. Concrete adapters inherit this boundary and then implement the protocol-specific mechanics for where vault collateral is held or invested.

Appendix: Shared Adapter Flow

During initialization, the adapter validates that the vault is registered in VaultFactory, stores the vault address, grants the vault an unlimited allowance for the vault asset, and runs adapter-specific initialization through _initialize. The approval is what lets VaultV2.push pull collateral back after the delegator has deallocated from the adapter.

allocate, deallocate, and requestDeallocate can only be called by the vault's configured delegator. Concrete adapters implement totalAssets, _allocate, and _deallocate. requestDeallocate is a no-op unless the concrete adapter overrides _requestDeallocate, so delayed withdrawal support is adapter-specific rather than guaranteed by the base class.

Privileged Functions

- allocate
- deallocate
- requestDeallocate

Core Invariants:

INV 1: An adapter must be bound to exactly one registered vault.

INV 2: Only the vault's configured delegator can allocate, deallocate, or request deallocation.

INV 3: freeAssets must represent vault collateral held directly by the adapter.

INV 4: totalAssets must represent all vault collateral value managed by the adapter.

INV 5: deallocate must report only assets that the vault can pull after the call.

INV 6: Delayed deallocation is only meaningful for adapters that override the base no-op request hook.

Issue_40	Standing max approvals to integrations
Severity	**Low**
Description	Adapters set forceApprove(integration, type(uint256).max) at init, leaving a standing allowance. Any issues in the future with the adapters expose them for their entire balance.
Recommendations	Consider giving approval only when required on a per-interaction basis, and resetting them to 0 once done.
Comments / Resolution	Acknowledged.

Issue_41	No recovery function
Severity	**Informational**
Description	The Adapter contract lacks a recovery/sweep function. Thus any tokens other than the asset stuck here would be unrecoverable.
Recommendations	Consider adding a sweep function so that admins can rescue non-asset tokens if required.
Comments / Resolution	Acknowledged.

Issue_42	deallocate can return more than the intended amount
Severity	Informational
Description	The deallocate function returns the current free asset amount as well as the deallocated amount. *return curFreeAssets + (curFreeAssets < amount ? _deallocate(amount - curFreeAssets) : 0);* So if the result of this function is ever used to deduct from some total value, it can lead to an underflow. This is not the case in the contracts in scope, but can very easily lead to an issue if the return value is ever used in an unsafe manner in the delegator contract. While this is not an issue now, future updates can trip this if not carefully documented.
Recommendations	Consider capping the return value to the input amount, or clearly documenting this so that this does not become an issue in a future adapter implementation.
Comments / Resolution	Acknowledged.

AppAdapter

AppAdapter is a vault adapter for one network, subnetwork, and operator guarantee position. It holds the vault asset directly and tracks which portion remains slashable during a configured duration window. Unlike external yield adapters, its main job is to model slashable guarantee exposure and delayed release rather than deposit collateral into another protocol.

Appendix: Stake, Slashing, and Delayed Deallocation

totalAssets is the vault asset balance held by the adapter. freeAssets is totalAssets minus slashable stake. Each allocation creates a new stake checkpoint. Delayed deallocation is tracked as debt that matures after duration, while immediate _deallocate returns zero. This means withdrawal availability depends on the debt schedule, slash state, and whether the protected duration has elapsed.

slash can only be called by the configured network middleware. It caps the slash by slashable stake, records the slash, decreases delegator limits, transfers assets to the burner, and calls the burner hook.

release can be called by the network or middleware. It records the stake as no longer slashable and decreases the delegator's limits to avoid automatic refill. Both slash and release affect the economic meaning of the adapter balance without using the normal external-protocol withdraw flow.

Privileged Functions

- slash
- release
- allocate
- deallocate
- requestDeallocate

Core Invariants:

INV 1: Initialization must set a nonzero operator, subnetwork, duration, and burner.

INV 2: totalAssets must equal the vault asset balance held by the adapter.

INV 3: freeAssets must equal totalAssets minus current slashable stake.

INV 4: Only network middleware can slash stake.

INV 5: Only the network or middleware can release slashable stake.

INV 6: Slashes must be recorded and transferred to the burner.

INV 7: Delayed deallocation debt must not become non-slashable before duration elapses.

INV 8: Slashing or release should reduce delegator limits so the guarantee is not immediately refilled.

Issue_43	Stale absolute AppAdapter debt can ignore or under-release later queued stake
Severity	High
Description	AppAdapter stores delayed deallocation debt as an absolute amount against the original stake checkpoint. When matured free assets are swept back to the vault, the adapter does not reduce the original stake baseline or otherwise mark the swept principal as completed. Later requests are compared directly against the old debt.latest() value. If the later request is smaller than the old debt, AppAdapter ignores it and never starts a new delay window. If the later request is larger, it records the larger absolute target, but after the second maturity it can under-release and leave a smaller tail stuck because that tail is below the old debt.latest(). Queued withdrawers can remain pending indefinitely even though the relevant delay windows have elapsed, and remaining stake can stay slashable when it should have been released.
Recommendations	Do not compare new deallocation requests directly against a stale absolute debt checkpoint after matured assets have already been swept. Track debt cumulatively relative to a synchronized stake baseline, or reset the stake and debt accounting whenever matured free AppAdapter assets are pushed back to the vault.
Comments / Resolution	Fixed.

Issue_44	Just-in-time deposits can hijack rewards
Severity	**Low**
Description	The AppAdapter can receive staking rewards deposited to the contract. Any user can frontrun such a reward distribution with a large deposit in order to capture most of the gains. Since staking rewards are discrete, users can deposit a large sum before rewards are notified and then immediately withdraw their deposit, and capture most of the staking reward given to the adapter.
Recommendations	Consider dripping staking rewards to the adapter over time, instead of in discrete chunks.
Comments / Resolution	Acknowledged.

Issue_45	AppAdapter delayed debt can desynchronize from current withdrawal demand
Severity	Low
Description	UniversalDelegator can attempt to clear obsolete AppAdapter delayed debt by calling requestDeallocate(0) after queued withdrawal demand has been satisfied elsewhere. AppAdapter translates this through current limit and slashable-state logic. If current limits are below current slashable stake, the reset may not clear the old debt. That stale debt can later mature and make stake free even though there is no remaining queue demand that justified the release. The monotonic debt model can also over-release stake when a previously large pending request later shrinks before maturity, because smaller follow-up requestDeallocate() calls do not reduce the existing debt checkpoint. Conversely, when requestDeallocate(0) does take the reset branch, it creates a fresh stake checkpoint with no carried-forward debt. If a privileged adapter reorder causes pending demand to be reassigned to another adapter before the original debt matures, the earlier AppAdapter release timer can be discarded and the same withdrawal demand can be rescheduled later on the new route. This can postpone exit from the slashable stake, but it depends on privileged route changes rather than an unprivileged attacker path.
Recommendations	Synchronize AppAdapter delayed debt with current pending withdrawal demand independently of the current allocation limit. When demand falls or disappears, allow stale debt to be reduced or cleared safely. When stake accounting is reset, either carry still-future debt into the fresh checkpoint or only allow the reset when no future-effective debt is outstanding.
Comments / Resolution	Acknowledged.

Issue_46	slash() may not quarantine future allocations into a slashed guarantee
Severity	Low
Description	AppAdapter.slash() decreases delegator limits by the slashed amount. Under unlimited absolute limits or max-share limits, reducing limits by only the realized slash amount may still leave meaningful allocation capacity for the same adapter. If the intended response to a slash is to stop new exposure to that guarantee, the current limit adjustment may be insufficient.
Recommendations	Clarify whether slashed AppAdapter guarantees should remain allocatable. If new allocations should stop after a slash, set the adapter's absolute and share limits to the desired post-slash exposure cap, or explicitly zero them until governance or middleware re-enables allocation.
Comments / Resolution	Acknowledged.

Issue_47	release() can reopen capacity after delayed debt exits
Severity	Low
Description	AppAdapter.release() sets delegator limits from the current slashable amount. Current slashable stake can still include assets that already have delayed deallocation debt scheduled but have not yet matured. After that delayed debt matures and is swept out, the limit can remain higher than the remaining durable stake. Later deposits can refill the adapter up to that stale limit, even though the release path appears intended to stop new allocations.
Recommendations	Set post-release limits against durable post-debt stake instead of current slashable stake. Alternatively, document that release() preserves a target guarantee size and may allow replacement deposits after delayed debt exits.
Comments / Resolution	Acknowledged.

MorphoVaultV2Adapter allocates vault collateral into one configured Morpho Vault V2. totalAssets is free collateral held by the adapter plus the assets previewed from redeeming the adapter's Morpho vault shares. It relies on Morpho ERC4626 share accounting to report the value of the external position back to VaultV2 through the delegator.

Initialization validates that the Morpho vault is recognized by the configured Morpho factory, uses the required adapter registry, has abdicated the adapter registry setter, and has the same asset as the VaultV2 collateral. These checks are intended to bind the adapter to a Morpho vault whose configuration cannot later drift away from the expected registry and asset assumptions.

Appendix: Morpho Allocation and Deallocation

_allocate deposits collateral into Morpho through a self-call helper. If the deposit succeeds and mints nonzero shares, the adapter reports the full amount as allocated. If it fails, allocation returns zero.

_deallocate caps withdrawal by preview-redeemable shares and estimated Morpho-side liquidity, then attempts withdraw in a try-catch. It only reports a positive amount when withdraw succeeds. If Morpho liquidity or view calls do not match actual withdrawability, vault withdrawals that depend on this adapter can be delayed or fail to recover expected assets.

Privileged Functions

- allocate
- deallocate
- requestDeallocate
- deposit

Core Invariants:

INV 1: The configured Morpho vault must be valid, immutable for registry purposes, and asset-compatible with the vault.
INV 2: totalAssets must equal free adapter collateral plus preview-redeemable Morpho assets.
INV 3: Allocation must only report success after a successful Morpho deposit.
INV 4: Deallocation must only report assets actually withdrawn to the adapter.
INV 5: Morpho view, liquidity, and withdrawal assumptions must remain valid for vault accounting and liveness.
INV 6: Only the vault's configured delegator can use inherited allocation and deallocation entry points.

Issue_48	Morpho liquidity adapter accounting assets can overstate executable withdrawal liquidity
Severity	Medium
Description	MorphoVaultV2Adapter._deallocate() limits the requested withdrawal by the Morpho vault's idle asset balance plus the configured Morpho liquidity adapter's realAssets(). This treats liquidityAdapter.realAssets() as executable withdrawal liquidity, but for Morpho-backed liquidity adapters realAssets() is an accounting NAV. For example, a Morpho Market V1 adapter can report expected supplied assets even when most of those assets are borrowed from the underlying Morpho Blue market. Morpho Blue withdraw() then rejects withdrawals that would leave totalBorrowAssets greater than totalSupplyAssets, so the accounting value is not equivalent to currently withdrawable liquidity. As a result, the Symbiotic adapter can attempt a Morpho vault withdrawal that was pre-approved by accounting NAV but cannot execute against the underlying markets. The try/catch in _deallocate() converts that failure into a zero deallocation, so instant withdrawals or withdrawal queue fills can remain pending until real liquidity returns even though the adapter appeared to have sufficient assets.
Recommendations	Consider acknowledging that Morpho liquidity adapter realAssets() is an accounting estimate and not guaranteed executable withdrawal liquidity. If Morpho Vault V2 exposes a penalty-based force-deallocation path for otherwise illiquid positions, consider adding a controlled adapter fallback or documenting that Symbiotic only supports the normal Morpho withdraw path
Comments / Resolution	Acknowledged.

Issue_49	Morpho vault balance can overstate the deallocatable amount
Severity	**Medium**
Description	In the Morpho adapter, the raw asset balance in the Morpho vault is used to estimate the withdrawable amount. *IERC20(IERC4626(vault).asset()).balanceOf(morphoVault)* *+ (liquidityAdapter == address(0) ? 0 :* *IMorphoLiquidityAdapter(liquidityAdapter).realAssets())* However, all these funds might not be available for withdrawal. This is because any funds donated to the Morpho Vault will register in this value, but not be actually withdrawable. Morpho records the actual deposits in _totalAssets, which does not take into consideration any donations. Thus, the vault can hold 100 tokens, but only 60 can be withdrawable. The issue is that _deallocate will always try to withdraw the maximum amount possible, so it will keep reverting trying to withdraw the non-existent amount, and so no amount will be de-allocated at all.
Recommendations	Consider acknowledging this issue and documenting this behaviour, admin controlled force-deallocations might be required to free up funds sometimes. Otherwise, calculate the available liquidity using total assets - total borrowed.
Comments / Resolution	Acknowledged.

Issue_50	Mutable Morpho exit gates can strand counted adapter NAV
Severity	**Low**
Description	MorphoVaultV2Adapter.__initialize() validates Morpho's adapter registry settings, but it does not require the share-send or asset-receive gate setters to be abdicated. If a Morpho curator later configures sendSharesGate or receiveAssetsGate to deny the Symbiotic adapter, Morpho withdraw() can revert. Symbiotic still counts the adapter's Morpho shares through previewRedeem(balanceOf(adapter)) in totalAssets(), while _deallocate() catches the failed withdrawal and returns 0. Vault NAV can therefore include Morpho shares that are not currently executable, blocking withdrawals, queue fills, or adapter removal until the external gate configuration changes.
Recommendations	Consider acknowledging this issue and documenting that future Morpho gate changes can strand counted NAV.
Comments / Resolution	Acknowledged.

Issue_51	Morpho rewards can be stuck unless redirected before depositing
Severity	**Informational**
Description	Morpho Vault V2 can have rewards distributed through Merkl. Currently, MorphoVaultV2Adapter does not have functionality to claim or distribute those rewards once they accrue. Since the adapter is the depositor into the Morpho Vault V2, rewards may be attributed to the adapter address. Morpho's documentation states that if an integrator interacts with Morpho through a smart contract or address that cannot claim rewards, they should contact Merkl before depositing to have rewards redirected. Therefore, unless such redirection is configured, rewards attributed to the adapter may not benefit vault users.
Recommendations	Consider acknowledging this issue since code changes are not required.
Comments / Resolution	Acknowledged.

Issue_52	Morpho allocation rounding can leak dust to external Morpho shareholders
Severity	Informational
Description	MorphoVaultV2Adapter allocates assets by depositing into an external Morpho Vault V2. If the external vault share price is high or donation-inflated, deposit rounding can cause the adapter to receive fewer Morpho shares than an exact pro-rata calculation would imply. The rounded value remains in the external Morpho vault and benefits all Morpho shareholders, while VaultV2 has consumed the full requested asset amount. The adapter already reverts zero-share deposits, so this is not a material allocation-bypass issue under normal ERC4626 share granularity. It is best treated as expected ERC4626 rounding dust unless the configured Morpho vault has unusually coarse share precision.
Recommendations	Consider acknowledging the rounding behavior in integration documentation.
Comments / Resolution	Acknowledged.

Issue_53	Symbiotic settlements inherit Morpho Vault V2 maxRate pricing lag
Severity	**Informational**
Description	MorphoVaultV2Adapter values its position from the external Morpho Vault V2 previewRedeem() accounting. Morpho Vault V2 deliberately caps reported totalAssets growth by maxRate, so previewRedeem() can remain below the vault's underlying adapter NAV while the reported value catches up. Symbiotic VaultV2 deposits and withdrawal queue fills are both priced from the adapter-reported value. New entrants can therefore enter at the canonical rate-limited Morpho ERC4626 price and later share in Morpho catch-up value that economically existed before they entered. Similarly, if WithdrawalQueue.fill() burns queued shares while the Morpho value is still rate-limited, queued users settle at the current reported rate and later catch-up value remains with active VaultV2 holders.
Recommendations	Consider acknowledging that Symbiotic share pricing inherits Morpho Vault V2's maxRate-delayed NAV recognition. Document that new deposits and withdrawal queue fills can share or crystallize later Morpho catch-up value when the configured Morpho vault is materially rate-limited.
Comments / Resolution	Acknowledged.

AaveV3Adapter

AaveV3Adapter allocates vault collateral into one Aave V3 pool. It stores the reserve aToken for the vault asset and reports totalAssets as free adapter collateral plus the adapter's aToken balance. The adapter relies on Aave's reserve accounting to treat aToken balance as the vault's external asset exposure.

During initialization, the adapter asks the immutable Aave pool for the reserve aToken matching the vault asset and approves the pool to pull the asset. Initialization reverts if no aToken exists. After initialization, the adapter is permanently tied to that pool and reserve for the vault asset.

Appendix: Aave Allocation and Deallocation

_allocate supplies assets to Aave in a try-catch. It reports the full amount on success and zero on failure.

_deallocate caps withdrawal by the requested amount, adapter aToken balance, and Aave getVirtualUnderlyingBalance. It attempts withdraw only when the capped amount is positive and reports a positive amount only if withdraw succeeds. This makes deallocation liveness depend on Aave reserve liquidity and the adapter's interpretation of the virtual underlying balance.

Privileged Functions

- allocate
- deallocate
- requestDeallocate

Core Invariants:

INV 1: Initialization must bind a valid aToken for the vault asset.
INV 2: totalAssets must equal free adapter collateral plus aToken balance.
INV 3: Allocation must only report success after a successful Aave supply.
INV 4: Deallocation must only report assets actually withdrawn to the adapter.
INV 5: Aave reserve liquidity and virtual underlying balance assumptions must remain valid for vault accounting and liveness.
INV 6: Only the vault's configured delegator can use inherited allocation and deallocation entry points.

Issue_54	Aave interest is not accounted for
Severity	High
Description	Aave pays interest via its rebasing token, increasing its balance over time. The AaveAdapter now records its allocation via a totalATokens value, which is updated during allocation/deallocation. However, the issue is that this completely ignores the interest accrued and thus does not generate any yield for the vault anymore.
Recommendations	Interest via atokens need to be accounted for in the Aave adapter. Consider capping the atoken growth to some rate, so that interest and minor donation can be captured, but not drastic donations to change vault composition.
Comments / Resolution	Fixed by reverting the change that has been made to the AaveV3Adapter due to Issue_02. However, it should be noted that this reintroduces the Issue_02 attack surface for this adapter.

Issue_55	AaveV3Adapter can report allocatable capacity when the reserve cannot accept supply
Severity	Low
Description	AaveV3Adapter inherits the base adapter allocatable() behavior that reports effectively unlimited capacity. UniversalDelegator.allocatable() can therefore report capacity for Aave even when the Aave reserve is paused, frozen, supply-capped, or otherwise unable to accept a supply transaction. Allocation automation can receive optimistic capacity and then allocate less or zero in the real path.
Recommendations	Consider acknowledging that AaveV3Adapter.allocatable() is a limit-side estimate and does not account for external Aave reserve availability.
Comments / Resolution	Acknowledged.

Issue_56	Aave incentive rewards can accrue outside vault accounting
Severity	Informational
Description	AaveV3Adapter accounts for the underlying asset balance represented by its aTokens, but Aave incentive rewards can accrue as separate reward assets. The adapter does not expose a claim path or include those rewards in freeAssets() or totalAssets(). If rewards are enabled for the supplied reserve, non-underlying incentive yield can be stranded outside VaultV2 accounting.
Recommendations	Consider acknowledging that non-underlying Aave rewards are not part of adapter accounting. If reward capture is intended, add a controlled claim function and define whether claimed rewards are distributed, swapped into the underlying asset, or swept by governance.

Comments / Resolution	Acknowledged.

Issue_57	Aave pool address is hardcoded
Severity	**Informational**
Description	The address of the aave pool is stored as an immutable and set during contract creation. Aave docs, however, recommend fetching the pool address on-demand from the PoolAddressesProvider contract instead.
Recommendations	Consider fetching the address from the PoolAddressesProvider contract whenever required.
Comments / Resolution	Acknowledged.

MigratablesFactory

MigratablesFactory is the shared versioned factory for upgradeable protocol entities. It stores whitelisted implementations as 1-indexed versions, deploys MigratableEntityProxy instances, registers them as entities, initializes them, and controls their migration path.

Appendix: Versioned Deployment and Migration

The owner appends implementations through whitelist. Each implementation must have FACTORY equal to this factory. Anyone can create an entity for a valid version. The factory deploys a proxy, registers it, and calls initialize with the supplied owner and data.

Only the current entity owner can migrate a registered entity, and only to a strictly newer version. blacklist marks a version as unsafe but does not block create or migrate, so it is not an enforcement gate.

Privileged Functions

- whitelist
- blacklist

Core Invariants:

INV 1: Only the factory owner can whitelist implementations or blacklist versions.

INV 2: A whitelisted implementation must have FACTORY equal to this factory.

INV 3: Version numbers must be nonzero and no greater than lastVersion.

INV 4: Every created proxy must be registered before initialization.

INV 5: Migration must only apply to registered entities and only be callable by the entity owner.

INV 6: Migration must only move to a strictly newer version.

INV 7: Blacklisted versions remain deployable and migratable.

Issue_58	Factory-created proxies can be whitelisted as implementations
Severity	**Low**
Description	MigratablesFactory.whitelist() verifies that a candidate implementation reports the expected FACTORY(). A factory-created proxy delegates that call to its implementation, so the proxy can satisfy the check even though it is proxy runtime code rather than a real implementation contract. If such a proxy is whitelisted as a new version, create() or migrate() to that version can deploy proxies whose implementation points to another proxy and fail or behave unexpectedly.
Recommendations	Reject known proxy bytecode during whitelist(), or require implementations to expose an immutable self-check that cannot be satisfied through delegatecall from a proxy. Also consider adding a deployment-time test that whitelisted versions can successfully initialize through a fresh proxy.
Comments / Resolution	Acknowledged.

Issue_59	Mutable totalEntities() is part of create() address derivation
Severity	**Informational**
Description	MigratablesFactory.create() salts include the mutable totalEntities() value. Because creates can be permissionless, another caller can increment totalEntities() before an expected create and shift the deterministic address that a user or integration precomputed. This can break address-dependent setup flows or allow a caller to occupy the next expected direct-factory address.
Recommendations	Consider acknowledging that direct factory create() addresses are not stable against intervening permissionless creates. Furthermore, consider documenting that create() addresses are only predictable while no intervening create() can occur.
Comments / Resolution	Acknowledged.

Issue_60	Entities can be initialized with a zero owner and become non-migratable
Severity	**Informational**
Description	MigratableEntity.initialize() permits owner_ to be address(0). Child entities that do not reject a zero owner can be created and registered with owner() permanently set to zero. Because migration is owner-driven, a zero-owner entity can become impossible to migrate through MigratablesFactory.
Recommendations	Consider acknowledging that zero-owner entities are non-migratable through MigratablesFactory. If zero-owner entities are not intended, reject address(0) as owner_ in MigratableEntity.initialize() or require every child entity to enforce a nonzero owner during its own initialization.
Comments / Resolution	Acknowledged.